

```

import os
import json
import hashlib
import uuid
from datetime import datetime, timedelta
from fastapi import FastAPI, HTTPException, Depends, Request, UploadFile, File, Form
from fastapi.responses import HTMLResponse, RedirectResponse, JSONResponse
from fastapi.staticfiles import StaticFiles
from fastapi.templating import Jinja2Templates
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials
from pydantic import BaseModel
from typing import Optional, List
import sqlite3
import groq
import shutil
from pathlib import Path
import aiofiles
import PyPDF2
from docx import Document

# ----- Configuration -----
SECRET_KEY = "your-super-secret-key-change-me"
GROQ_API_KEY = os.getenv("GROQ_API_KEY") or "your-groq-api-key"
UPLOAD_DIR = Path("uploads")
UPLOAD_DIR.mkdir(exist_ok=True)
DB_PATH = "asifeda.db"
MAX_STORAGE_PER_USER = 100 * 1024 * 1024 # 100 MB

# ----- Database Setup -----
def init_db():
    conn = sqlite3.connect(DB_PATH)
    c = conn.cursor()
    c.execute("CREATE TABLE IF NOT EXISTS users (

```

```
id INTEGER PRIMARY KEY AUTOINCREMENT,  
email TEXT UNIQUE NOT NULL,  
full_name TEXT NOT NULL,  
password_hash TEXT NOT NULL,  
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
storage_used INTEGER DEFAULT 0  
)")
```

```
c.execute("CREATE TABLE IF NOT EXISTS files (  
id INTEGER PRIMARY KEY AUTOINCREMENT,  
user_id INTEGER NOT NULL,  
filename TEXT NOT NULL,  
original_name TEXT NOT NULL,  
file_path TEXT NOT NULL,  
file_size INTEGER NOT NULL,  
uploaded_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
processed BOOLEAN DEFAULT 0,  
FOREIGN KEY (user_id) REFERENCES users (id) ON DELETE CASCADE  
)")
```

```
c.execute("CREATE TABLE IF NOT EXISTS pyqs (  
id INTEGER PRIMARY KEY AUTOINCREMENT,  
user_id INTEGER NOT NULL,  
topic TEXT NOT NULL,  
question TEXT NOT NULL,  
options TEXT NOT NULL,  
correct_answer INTEGER NOT NULL,  
explanation TEXT,  
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
)")
```

```
c.execute("CREATE TABLE IF NOT EXISTS notes (  
id INTEGER PRIMARY KEY AUTOINCREMENT,  
user_id INTEGER NOT NULL,  
title TEXT NOT NULL,  
content TEXT NOT NULL,
```

```

created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
)"""
c.execute("""CREATE TABLE IF NOT EXISTS conversations (
id INTEGER PRIMARY KEY AUTOINCREMENT,
user_id INTEGER NOT NULL,
role TEXT NOT NULL,
content TEXT NOT NULL,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
)""")
c.execute("""CREATE TABLE IF NOT EXISTS predictions (
id INTEGER PRIMARY KEY AUTOINCREMENT,
user_id INTEGER NOT NULL,
topic TEXT NOT NULL,
predicted_questions TEXT NOT NULL,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
)""")
conn.commit()
conn.close()

```

```

init_db()

```

```

# ----- FastAPI App -----

```

```

app = FastAPI(title="ASIFEDA AI", version="5.0")

```

```

# For serving HTML (we'll embed frontend inside index.html)

```

```

templates = Jinja2Templates(directory="templates")

```

```

os.makedirs("templates", exist_ok=True)

```

```

# Write the frontend HTML to a template file

```

```

with open("templates/index.html", "w", encoding="utf-8") as f:

```

```

f.write("""

```



Email
Password
Login / Sign Up

First time? Account created automatically.

""")

----- API Endpoints -----

security = HTTPBearer()

def get_db():

conn = sqlite3.connect(DB_PATH)

conn.row_factory = sqlite3.Row

try:

yield conn

finally:

conn.close()

```
def get_current_user(credentials: HTTPAuthorizationCredentials = Depends(security),
```

```
db=Depends(get_db)):
```

```
token = credentials.credentials
```

```
# Simple JWT verification (simplified – in production use python-jose)
```

```
try:
```

```
payload = json.loads(token) # dummy
```

```
user_id = payload.get("user_id")
```

```
user = db.execute("SELECT * FROM users WHERE id = ?", (user_id,)).fetchone()
```

```
if user:
```

```
return user
```

```
except:
```

```
pass
```

```
raise HTTPException(401, "Invalid token")
```

```
class LoginData(BaseModel):
```

```
email: str
```

```
password: str
```

```
@app.post("/api/auth/login")
```

```
def login(data: LoginData, db=Depends(get_db)):
```

```
email = data.email
```

```
password = data.password
```

```
user = db.execute("SELECT * FROM users WHERE email = ?", (email,)).fetchone()
```

```
if not user:
```

```
# Auto-register
```

```
pwd_hash = hashlib.sha256(password.encode()).hexdigest()
```

```
db.execute("INSERT INTO users (email, full_name, password_hash) VALUES (?, ?, ?)",
```

```
(email, email.split('@')[0], pwd_hash))
```

```
db.commit()
```

```
user = db.execute("SELECT * FROM users WHERE email = ?", (email,)).fetchone()
```

else:

if user["password_hash"] != hashlib.sha256(password.encode()).hexdigest():

raise HTTPException(401, "Invalid password")

Create token (simplified)

token = json.dumps({"user_id": user["id"], "exp": (datetime.utcnow() +
timedelta(days=7)).isoformat()})

return {"access_token": token, "token_type": "bearer"}

@app.get("/api/users/stats")

def stats(user=Depends(get_current_user), db=Depends(get_db)):

storage = db.execute("SELECT SUM(file_size) as total FROM files WHERE user_id = ?",
(user["id"],)).fetchone()["total"] or 0

pyq_count = db.execute("SELECT COUNT(*) FROM pyqs WHERE user_id = ?",
(user["id"],)).fetchone()[0]

notes_count = db.execute("SELECT COUNT(*) FROM notes WHERE user_id = ?",
(user["id"],)).fetchone()[0]

return {"storage_used": storage, "storage_used_mb": storage/1024/1024, "pyq_count":
pyq_count, "notes_count": notes_count}

@app.post("/api/ai/chat")

async def chat(request: Request, user=Depends(get_current_user), db=Depends(get_db)):

body = await request.json()

msg = body.get("message")

RAG: search user's files

files = db.execute("SELECT file_path FROM files WHERE user_id = ?", (user["id"],)).fetchall()

context = ""

for f in files:

if f["file_path"].endswith(".txt"):

try:

with open(f["file_path"], "r") as file:

text = file.read()

if msg.lower() in text.lower():

context += f"\n[From {f['file_path']}]: {text[:1500]}\n"

```

except: pass
client = groq.Groq(api_key=GROQ_API_KEY)
sys_prompt = f"You are ASIFEDA AI exam tutor. Use this context if relevant: {context}"
response = client.chat.completions.create(
    model="llama-3.3-70b-versatile",
    messages=[{"role": "system", "content": sys_prompt}, {"role": "user", "content": msg}],
    max_tokens=1500
)
reply = response.choices[0].message.content
return {"reply": reply}

```

```

@app.post("/api/ai/generate-quiz")
async def generate_quiz(request: Request, user=Depends(get_current_user)):
    body = await request.json()
    topic = body.get("topic")
    exam = body.get("exam")
    count = body.get("count")
    client = groq.Groq(api_key=GROQ_API_KEY)
    prompt = f"Generate {count} MCQs on '{topic}' for {exam} exam. Return JSON array: [{{'q': 'question', 'opts': ['a', 'b', 'c', 'd'], 'ans': 0, 'exp': 'explanation'}}]"
    response = client.chat.completions.create(
        model="llama-3.3-70b-versatile",
        messages=[{"role": "user", "content": prompt}],
        temperature=0.7
    )
    import json
    try:
        questions = json.loads(response.choices[0].message.content.replace("`json", "").replace("`", ""))
    except:
        questions = []
    return {"questions": questions}

```

```

@app.post("/api/ai/predict-future")

```

```

async def predict_future(request: Request, user=Depends(get_current_user)):
    body = await request.json()
    topic = body.get("topic")
    client = groq.Groq(api_key=GROQ_API_KEY)
    prompt = f"Predict 10 most likely exam questions on '{topic}' for UPSC 2025. Return JSON array of strings."
    response = client.chat.completions.create(
        model="llama-3.3-70b-versatile",
        messages=[{"role": "user", "content": prompt}]
    )
    import json
    preds = json.loads(response.choices[0].message.content.replace("`json", "").replace("`", ""))
    return {"predictions": preds}

```

```

@app.post("/api/pyq/generate-batch")
async def generate_pyq_batch(request: Request, user=Depends(get_current_user),
db=Depends(get_db)):
    body = await request.json()
    topic = body.get("topic")
    count = body.get("count", 500)
    client = groq.Groq(api_key=GROQ_API_KEY)
    all_pyqs = []
    for _ in range(0, count, 20):
        batch = min(20, count - len(all_pyqs))
        prompt = f"Generate {batch} previous year style questions on '{topic}' for UPSC. Return JSON array: [{ {'question': '...', 'options': ['a', 'b', 'c', 'd'], 'correct': 0, 'explanation': '...' } }]"
        resp = client.chat.completions.create(model="llama-3.3-70b-versatile", messages=
[{"role": "user", "content": prompt}])
        data = json.loads(resp.choices[0].message.content.replace("`json", "").replace("`", ""))
        for item in data:
            db.execute("INSERT INTO pyqs (user_id, topic, question, options, correct_answer, explanation)
VALUES (?, ?, ?, ?, ?)",
            (user["id"], topic, item["question"], json.dumps(item["options"]), item["correct"],

```

```

item.get("explanation",""))
db.commit()
all_pyqs.append(item)
return {"added": len(all_pyqs)}

```

```

@app.get("/api/pyq/search")
def search_pyq(topic: str, user=Depends(get_current_user), db=Depends(get_db)):
    rows = db.execute("SELECT * FROM pyqs WHERE user_id = ? AND topic LIKE ?",
    (user["id"], f"%{topic}%")).fetchall()
    return [{"question": r["question"], "options": r["options"], "correct_answer": r["correct_answer"],
    "explanation": r["explanation"]} for r in rows]

```

```

@app.post("/api/files/upload")
async def upload_files(files: List[UploadFile] = File(...), user=Depends(get_current_user),
db=Depends(get_db)):
    # Check quota
    used = db.execute("SELECT SUM(file_size) as total FROM files WHERE user_id = ?",
    (user["id"],)).fetchone()["total"] or 0
    for f in files:
        if used + f.size > MAX_STORAGE_PER_USER:
            raise HTTPException(400, "Storage quota exceeded (100MB)")
        file_path = UPLOAD_DIR / f"{uuid.uuid4()}_{f.filename}"
        async with aiofiles.open(file_path, "wb") as out:
            content = await f.read()
            await out.write(content)
        db.execute("INSERT INTO files (user_id, filename, original_name, file_path, file_size) VALUES
        (?, ?, ?, ?, ?)",
        (user["id"], file_path.name, f.filename, str(file_path), len(content)))
        db.commit()
        used += len(content)
    return {"message": "Uploaded"}

```

```

@app.get("/api/files/list")

```

```
def list_files(user=Depends(get_current_user), db=Depends(get_db)):
    rows = db.execute("SELECT id, original_name, file_size FROM files WHERE user_id = ?",
    (user["id"],)).fetchall()
    return [{"id": r["id"], "original_name": r["original_name"], "file_size": r["file_size"]} for r in
    rows]
```

```
@app.get("/")
async def root():
    with open("templates/index.html", "r") as f:
        return HTMLResponse(content=f.read())
```

```
if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8000)
```